

Project Report

Student Event Management System

Student Name: M J C Thathsarani

Student ID: 23IT0538

Module: IT1208 - Web Technologies

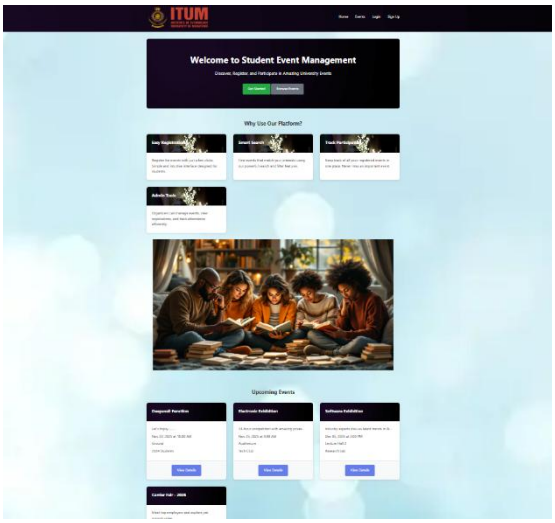
1. System Overview

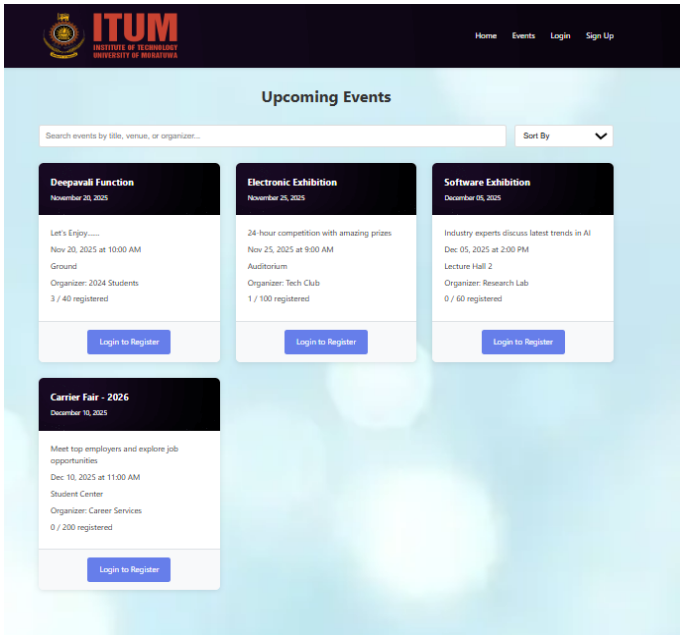
The **Student Event Management System** is a web-based application that helps university students organize and register for events such as workshops, seminars, and hackathons.
It uses the **XAMPP/LAMP stack** (Apache, MySQL, PHP) with **HTML, CSS, and JavaScript** for the front end.

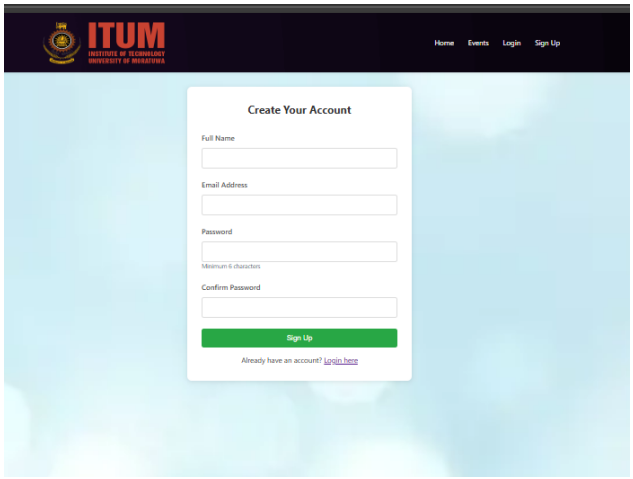
1.1 Key Features

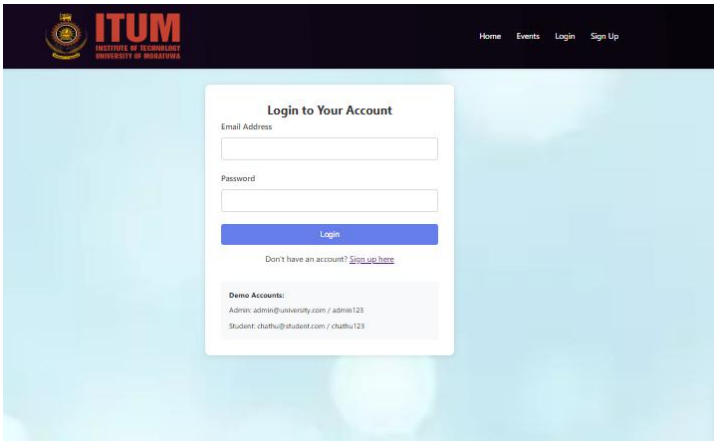
- **User Login & Registration:** Secure system using PHP sessions and password hashing (BCrypt).
- **Responsive Design:** Works well on phones, tablets, and computers.
- **Event Registration:** Logged-in students can view and register for available events.

1.2 System Screenshots



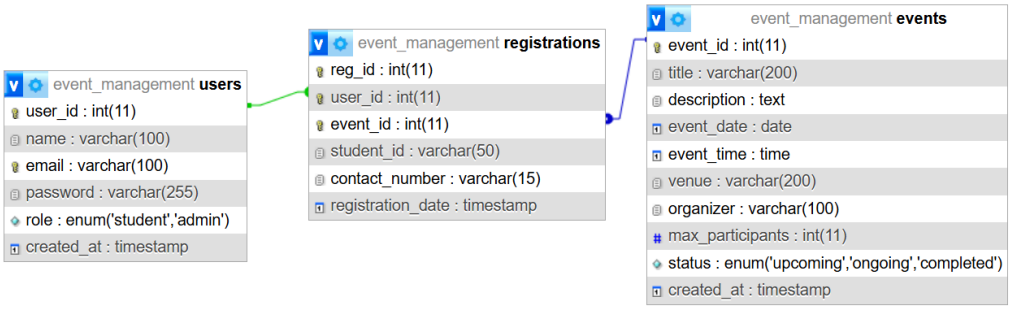






2. Database Schema

The system has **three main tables:** users, events, and registrations.



All database operations use **PHP PDO** for security and prepared statements.

2.1 Tables Overview

Table	purpose
users	Stores student login
events	Stores event details
registration	Links students to the registered events

2.2 Table Details

users

- user_id (INT, PK)
- name (VARCHAR)
- email (VARCHAR, UNIQUE)
- password_hash (VARCHAR)
- role (ENUM: 'student', 'admin')
- created_at(DATETIME)

events

- event_id (INT, PK)
- title (VARCHAR)
- description(VARCHAR)
- event_date (DATE)
- event_time (DATETIME)
- venue (VARCHAR)
- organizer(TEXT)
- max_participants(INT)
- status(VARCHAR)
- created_at(DATETIME)

registrations

- reg_id (INT, PK)
- user_id (INT, FK to users)
- event_id (INT, FK to events)
- student_id(INT)
- contact_number(INT)
- registration_date(DATE)

3. Core Functionalities

3.1 User Authentication

- During registration, passwords are hashed using password_hash() with BCRYPT.
- During login, password_verify() checks if the password is correct.
- On success, PHP stores user_id and role in $_SESSION.

3.2 Event Registration

1. Check if the user is logged in and is a student.
2. Check if already registered for the same event.
3. Insert record using a **PDO prepared statement** to prevent SQL injection.

3.3 Client-side Validation

JavaScript checks form fields before submission — for example, email format and required fields — improving the user experience.

4. Reflection

4.1 Learning Outcomes

- **HTML/CSS:** Created a mobile-friendly layout.
- **PHP Sessions:** Used for login and user roles (admin/student).
- **MySQL + PDO:** Built a secure database with prepared statements.

4.2 Challenges

- SQL syntax errors in queries.
- Fixing undefined variables in PHP.
- Managing correct file paths (e.g., require_once('../config.php')).